UNIVERSIDADE DE CAXIAS DO SUL
ÁREA DO CONHECIMENTO DE CIÊNCIAS EXATAS E ENGENHARIAS

LORENZO MOTTIN JABLONSKI

FILTROS DE BLOOM PARA OTIMIZAÇÃO DE I/O EM BIG DATA: ESTUDO COMPARATIVO E PROPOSTA DE UM MODELO HÍBRIDO

CAXIAS DO SUL

2026

LORENZO MOTTIN JABLONSKI

FILTROS DE BLOOM PARA OTIMIZAÇÃO DE I/O EM BIG DATA: ESTUDO COMPARATIVO E PROPOSTA DE UM MODELO HÍBRIDO

Trabalho de Conclusão de Curso I apresentado como requisito parcial à obtenção do título de Bacharel em Ciência da Computação na Área do Conhecimento de Ciências Exatas e Engenharias da Universidade de Caxias do Sul.

Orientador: Prof. Helena Graziottin Ribeiro

CAXIAS DO SUL

2026

LORENZO MOTTIN JABLONSKI

FILTROS DE BLOOM PARA OTIMIZAÇÃO DE I/O EM BIG DATA: ESTUDO COMPARATIVO E PROPOSTA DE UM MODELO HÍBRIDO

Trabalho de Conclusão de Curso I apresentado como requisito parcial à obtenção do título de Bacharel em Ciência da Computação na Área do Conhecimento de Ciências Exatas e Engenharias da Universidade de Caxias do Sul.

Aprovado em ___/___/2026

BANCA EXAMINADORA

_____

Prof. Helena Graziottin Ribeiro
Universidade de Caxias do Sul - UCS

_____

Prof. Daniel Luis Notari
Universidade de Caxias do Sul - UCS

_____

Prof. Ricardo Vargas Dorneles
Universidade de Caxias do Sul - UCS

RESUMO

A crescente geração de dados na era digital impõe desafios severos às arquiteturas computacionais, especialmente no que tange ao gargalo de operações de entrada e saída (I/O) em dispositivos de armazenamento secundário. Estruturas de indexação determinísticas, embora precisas, tornam-se proibitivas em cenários de Big Data devido ao elevado consumo de memória RAM. Neste contexto, esta pesquisa aborda a otimização de estruturas de dados probabilísticas, com foco no Filtro de Bloom e suas variantes, como mecanismo de triagem preliminar para mitigar consultas mal-sucedidas em bancos de dados de larga escala. O objetivo geral é implementar e avaliar empiricamente uma arquitetura otimizada baseada no Filtro de Bloom, integrando técnicas avançadas como o modelo r-dimensional matricial particionado com contagem e algoritmos de espalhamento eficientes. A metodologia adotada é de natureza aplicada, com abordagem quantitativa e método experimental, compreendendo a implementação em linguagem C (padrão ANSI C99) de cinco modelos — Filtro Padrão (SBF), de Contagem (CBF), Particionado (PBF), r-Dimensional (rDBF) e um modelo Híbrido —, além de uma linha de base determinística via `uthash`. Os experimentos previstos utilizarão a função de espalhamento MurmurHash2 e conjuntos de dados sintéticos (UUIDs, CPFs, CNPJs e URLs) para medir consumo de memória, Taxa de Falsos Positivos (FPR) prática versus teórica, e latências de inserção e consulta. Como resultados esperados, almeja-se validar a eficácia do Filtro de Bloom como camada de triagem de alta velocidade, comprovar a convergência entre FPR prática e teórica e mapear os limites de viabilidade do modelo híbrido proposto.

Palavras-chave: Filtro de Bloom. Estruturas de Dados Probabilísticas. Big Data. Taxa de Falsos Positivos. Otimização de Memória. Indexação.

ABSTRACT

The exponential growth of data in the digital age imposes severe challenges on computer architectures, particularly regarding the input/output (I/O) bottleneck in secondary storage devices. Deterministic indexing structures, although accurate, become prohibitive in Big Data scenarios due to high RAM consumption. In this context, this research addresses the optimization of probabilistic data structures, focusing on the Bloom Filter and its variants as a preliminary screening mechanism to mitigate unsuccessful queries in large-scale databases. The general objective is to implement and empirically evaluate an optimized architecture based on the Bloom Filter, integrating advanced techniques such as the partitioned r-dimensional matrix model with counting and efficient hashing algorithms. The methodology is applied in nature, with a quantitative approach and experimental method, comprising the implementation in C language (ANSI C99 standard) of five models — Standard Bloom Filter (SBF), Counting Bloom Filter (CBF), Partitioned Bloom Filter (PBF), r-Dimensional Bloom Filter (rDBF), and a Hybrid model — plus a deterministic baseline via `uthash`. The planned experiments will use the MurmurHash2 hash function and synthetic datasets (UUIDs, CPFs, CNPJs, and URLs) to measure memory consumption, practical versus theoretical False Positive Rate (FPR), and insertion and lookup latencies. As expected results, we aim to validate the effectiveness of the Bloom Filter as a high-speed screening layer, confirm the convergence between practical and theoretical FPR, and map the viability limits of the proposed hybrid model.

Keywords: Bloom Filter. Probabilistic Data Structures. Big Data. False Positive Rate. Memory Optimization. Indexing.

LISTA DE FIGURAS

Figura 1 – Representação visual dos processos de inserção no filtro. 22
Figura 2 – Representação visual dos processos de consulta no filtro. 23

LISTA DE TABELAS

Tabela 1 – Comparação entre estruturas de dados probabilísticas 20
Tabela 2 – Cronograma de atividades previstas para o TCC II. 39

LISTA DE ABREVIATURAS E SIGLAS

BF	Filtro de Bloom (*Bloom Filter*)
CBF	Filtro de Bloom com Contagem (*Counting Bloom Filter*)
CDN	Rede de Distribuição de Conteúdo (*Content Delivery Network*)
CPF	Cadastro de Pessoas Físicas
CNPJ	Cadastro Nacional da Pessoa Jurídica
CPU	Unidade Central de Processamento (*Central Processing Unit*)
CS	*Content Store*
FN	Falso Negativo (*False Negative*)
FP	Falso Positivo (*False Positive*)
FPR	Taxa de Falsos Positivos (*False Positive Rate*)
FTDF	Filtro Bidimensional Rápido (*Fast Two Dimensional Filter*)
GCC	Coleção de Compiladores GNU (*GNU Compiler Collection*)
HDD	Disco Rígido Magnético (*Hard Disk Drive*)
I/O	Entrada e Saída (*Input/Output*)
NDN	Rede de Dados Nomeados (*Named Data Networking*)
NoSQL	*Not Only SQL*
NVMe	*Non-Volatile Memory Express*
PBF	Filtro de Bloom Particionado (*Partitioned Bloom Filter*)
PIT	*Pending Interest Table*
POSIX	*Portable Operating System Interface*
RAM	Memória de Acesso Aleatório (*Random Access Memory*)
rDBF	Filtro de Bloom r-Dimensional (*r-Dimensional Bloom Filter*)
SBF	Filtro de Bloom Padrão (*Standard Bloom Filter*)
SGBD	Sistema Gerenciador de Bancos de Dados

SSD	Unidade de Estado Sólido (*Solid State Drive*)
TCC	Trabalho de Conclusão de Curso
URL	Localizador Uniforme de Recursos (*Uniform Resource Locator*)
UUID	Identificador Único Universal (*Universally Unique Identifier*)
VI-CBF	Filtro de Bloom com Contagem e Incremento Variável (*Variable-Increment Counting Bloom Filter*)
VP	Verdadeiro Positivo (*True Positive*)
VN	Verdadeiro Negativo (*True Negative*)

SUMÁRIO

1 INTRODUÇÃO 11
1.1 Contextualização e Problema 11
1.2 Justificativa 12
1.3 Objetivos 13
1.4 Estrutura do Trabalho 13

2 FUNDAMENTAÇÃO TEÓRICA 14
2.1 Bases da Computação para Dados 14
2.1.1 Estruturas de Indexação e o Custo de I/O 15
2.1.2 Funções de Hash: Propriedades e Algoritmos 16
2.2 Estruturas de Dados Probabilísticas 17
2.2.1 Conceito e Trade-offs: Espaço vs. Precisão 18
2.2.2 Avaliação de Precisão: Matriz de Confusão e Métricas 18
2.2.3 Outras Estruturas do Estado da Arte 19

3 O FILTRO DE BLOOM 21
3.1 Funcionamento: Inserção e Consulta 21
3.2 Análise Matemática: Probabilidade de Falsos Positivos (FPR) 23
3.2.1 A Otimização: A Relação Ideal entre m, n e k 24
3.3 Limitações do Filtro Original 24
3.4 Variantes do Filtro de Bloom 25
3.4.1 Filtro de Bloom com Contagem (*Counting Bloom Filter)* 25
3.4.2 Filtro de Bloom Particionado (*Partitioned Bloom Filter)* 26
3.4.3 Filtro de Bloom r-Dimensional (rDBF) 27
3.5 Aplicações Modernas e Ferramentas 28
3.5.1 Casos de Uso na Indústria (Sistemas de Larga Escala) 28
3.5.2 Bibliotecas e Implementações Disponíveis 30

4 PROJETO DE DESENVOLVIMENTO 31
4.1 Arquitetura da Solução e Modelos Propostos 31
4.2 Tecnologias e Ferramentas de Desenvolvimento 33
4.3 Engenharia de Dados: Datasets e Estratégia de Amostragem 34
4.3.1 Natureza e Tipologia das Chaves de Teste 34
4.3.2 Mecanismo de Partição e Protocolo Experimental 35
4.4 Parâmetros de Teste e Métricas de Avaliação 36
4.4.1 Parâmetros de Configuração (Variáveis Independentes) 36

4.4.2 Métricas de Desempenho (Variáveis Dependentes) 36

5 CONSIDERAÇÕES FINAIS 38
5.1 Resultados Esperados 38
5.2 Cronograma de Atividades para o TCC II 39

REFERÊNCIAS 40

APÊNDICE A – DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL 42

1 INTRODUÇÃO

A presente pesquisa insere-se no contexto da otimização de estruturas de dados para sistemas de computação de alto desempenho, com foco específico nas abordagens probabilísticas para triagem e recuperação de informações em cenários de larga escala.

1.1 CONTEXTUALIZAÇÃO E PROBLEMA

A atual era digital é marcada pela geração exponencial de informações, frequentemente descrita como a "Era dos *Zettabytes*"(10^{21} *bytes*). À medida que o volume de dados cresce em uma escala sem precedentes, os mecanismos tradicionais de busca e armazenamento enfrentam desafios físicos e lógicos severos. Embora o poder de processamento das Unidades Centrais de Processamento (CPUs) tenha evoluído rapidamente, as operações de entrada e saída (I/O) em dispositivos de armazenamento secundário continuam a representar o principal gargalo de desempenho em sistemas computacionais.

Neste contexto, é imperativo compreender que em Sistemas Gerenciadores de Bancos de Dados (SGBDs) e em sistemas de arquivos estruturados, a busca por um registro específico exige a varredura e a leitura física de blocos no disco. Mesmo que os meios de armazenamento secundário contemporâneos (como os SSDs baseados em NVMe) sejam consideravelmente mais rápidos do que os discos magnéticos tradicionais (HDDs), o volume colossal de dados manipulados atualmente requer que tenhamos estruturas de acesso otimizadas. Se o sistema precisar acessar o disco físico todas as vezes para confirmar que um dado **não** existe, a latência acumulada tornará a aplicação inviável.

Historicamente, esse problema é mitigado pelo uso de índices e estruturas determinísticas, como *HashSets* ou árvores de busca (*B-Trees*), que mantêm mapeamentos exatos na memória principal (RAM). Contudo, em cenários de *Big Data*, reter a totalidade ou os índices exatos de um conjunto de dados massivo torna-se exponencialmente problemático, exaurindo a memória RAM disponível e forçando o uso de paginação do sistema operacional, o que degrada ainda mais a performance.

Visando solucionar esse problema computacional, em 1970, Burton H. Bloom publicou o trabalho seminal *"Space/Time Trade-offs in Hash Coding with Allowable Errors"* (BLOOM, 1970). Nele, Bloom apresentou métodos de codificação *hash* projetados para otimizar o uso de memória. A principal inovação residiu em uma quebra de paradigma: aceitar uma pequena margem controlável de erros (falsos positivos) em troca de uma redução drástica na área de armazenamento necessária.

Essa estrutura, que hoje conhecemos como Filtro de Bloom, atua essencialmente como

uma camada de triagem preliminar — um recurso de pré-indexação alocado inteiramente na memória RAM. Operando sobre um vetor compacto de *bits* e utilizando funções de espalhamento (*hash*) não criptográficas, o filtro permite verificar a inexistência de valores em um grupo em tempo constante $\mathcal{O}(k)$. Se o filtro afirmar que o dado não está presente, a custosa pesquisa no armazenamento secundário é sumariamente evitada.

Apesar de sua elegância arquitetônica, a implementação clássica do Filtro de Bloom impõe limitações significativas. A estrutura original é de tamanho fixo, sofrendo degradação de precisão à medida que atinge sua capacidade máxima; não suporta a exclusão de elementos; e possui um custo computacional linear associado ao processamento de múltiplas funções *hash*. Para contornar essas barreiras, a comunidade científica propôs inúmeras variantes ao longo das décadas. Destacam-se o *Counting Bloom Filter* (CBF) e o *Cuckoo Filter* para viabilizar exclusões; o *Scalable Bloom Filter* (SBF) para crescimento dinâmico de fatias; bem como arquiteturas matriciais de alta performance voltadas para redes, como o *Fast Two Dimensional Filter* (FTDF) e o modelo bidimensional rDBF, que otimizam o espalhamento e o consumo computacional. Diante da vasta gama de soluções fragmentadas, o problema central desta pesquisa reside em como integrar, aplicar e avaliar essas arquiteturas otimizadas — especialmente as matrizes dimensionais e o particionamento — para extrair a máxima eficiência com o menor índice de falhas em ambientes práticos.

1.2 JUSTIFICATIVA

O estudo e o aprimoramento de estruturas de dados probabilísticas são vitais para o cenário tecnológico atual. A escalabilidade irrestrita de servidores em nuvem atrelou diretamente o consumo excessivo de memória RAM e processamento (CPU) a elevados custos financeiros e energéticos. Algoritmos que otimizam essas frentes deixam de ser apenas um exercício matemático e passam a ser uma exigência de engenharia sustentável e econômica.

Além disso, esta pesquisa justifica-se pela consolidação de uma mudança de mentalidade na Ciência da Computação: desafiar o dogma de que "respostas computacionais devem ser sempre exatas". A tolerância ao erro controlado provou ser uma estratégia altamente rentável, evidenciada pela adoção massiva dessas estruturas por gigantes da tecnologia. Navegadores como o Google Chrome dependem de variantes do filtro em sua ferramenta *Safe Browsing* para proteger usuários de *phishing* sem sacrificar a banda de rede; e bancos de dados de classe mundial, como o Apache Cassandra e o Google BigTable, dependem fundamentalmente dessas estruturas para manterem suas operações em frações de segundo. Portanto, dominar e otimizar tais algoritmos alinha a presente pesquisa com o estado da arte do desenvolvimento de *software* em escala global.

1.3 OBJETIVOS

O objetivo geral desse trabalho é implementar e avaliar empiricamente uma arquitetura otimizada baseada no Filtro de Bloom, integrando técnicas avançadas de variação estrutural (como o modelo r-dimensional matricial particionado com contagem e algoritmos de espalhamento eficientes), com o intuito de mitigar as limitações clássicas da estrutura e demonstrar sua eficácia na economia de recursos computacionais.

Para o alcance do objetivo geral delineado, estipulam-se os seguintes objetivos específicos:

- Desenvolver a implementação do Filtro de Bloom clássico e de variantes estruturais na linguagem C, utilizando algoritmos de espalhamento não criptográficos velozes, como a família MurmurHash.
- Conceber uma arquitetura de testes (*scripts* e *datasets*) capaz de injetar dados em massa e mensurar rigorosamente o tempo de execução e a ocorrência prática de falsos positivos frente à probabilidade teórica.
- Quantificar e analisar o impacto da redução do número de chamadas de *hash* em modelos matriciais, particionados e mistos em comparação ao método tradicional, além de consumo de memória deles com e sem a aplicação de contagem.
- Comparar o consumo físico de memória (RAM) e o tempo de busca da solução probabilística implementada em contraste com estruturas determinísticas nativas tradicionais.

1.4 ESTRUTURA DO TRABALHO

Para apresentar o desenvolvimento desta pesquisa de forma lógica e sequencial, este documento está organizado em capítulos. Após esta introdução, o Capítulo 2 estabelece a fundamentação teórica, abordando os conceitos de indexação e as bases das estruturas de dados probabilísticas. O Capítulo 3 dedica-se à análise profunda do Filtro de Bloom, detalhando seu funcionamento matemático, taxas de erro e casos de uso na indústria e variações. O Capítulo 4 descreve a proposta de desenvolvimento a nível técnico. O Capítulo 5 apresenta as considerações finais do trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

No cenário contemporâneo da tecnologia da informação, os dados gerados por aplicações e usuários são organizados e gerenciados por Sistemas Gerenciadores de Bancos de Dados (SGBDs). Estes sistemas operam sobrepostos aos sistemas de arquivos nativos dos sistemas operacionais, os quais são responsáveis por coordenar a persistência física dos blocos de dados nos dispositivos de armazenamento secundário. À medida que o volume dessas informações cresce exponencialmente, a realização de buscas lineares ou varreduras completas nos arquivos de dados (*full table scans*) torna-se computacionalmente proibitiva devido ao alto custo de operações de entrada e saída (I/O). Para mitigar esse problema e acelerar a recuperação de informações, os bancos de dados utilizam estruturas auxiliares de acesso conhecidas como índices. (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

Os índices funcionam como mapeamentos lógicos compactos que apontam diretamente para a localização física de um registro no disco, evitando a necessidade de ler o conjunto de dados em sua totalidade. Na literatura de computação, existem diferentes tipologias de índices, cada qual adequada a um padrão específico de carga de trabalho. Entre as abordagens mais difundidas destacam-se os índices baseados em Árvores-B (e suas variantes como B+ e B*), amplamente empregados para consultas que envolvem intervalos de valores e ordenação, e os índices baseados em tabelas de espalhamento (*hash indexes*), projetados para buscas por igualdade exata de chave e que oferecem tempo de acesso constante.

Com o advento do Big Data, contudo, até mesmo o armazenamento e a manutenção dessas estruturas indexadas tradicionais passaram a impor desafios severos à memória principal dos servidores. Diante disso, este capítulo dedica-se a apresentar o arcabouço teórico que fundamenta as técnicas de indexação e os mecanismos de busca otimizados em larga escala. Inicialmente, serão discutidas as estruturas de indexação, funções de espalhamento (*hash*), mecanismos matemáticos essenciais para a distribuição de dados e construção de índices rápidos. Na sequência, abordam-se as estruturas de dados probabilísticas e as métricas estatísticas de avaliação de sua precisão, premeditando a análise aprofundada do Filtro de Bloom e de suas variantes arquitetônicas, as quais se posicionam como soluções de alto desempenho para o gerenciamento de pertinência em cenários de alta volumetria.

2.1 BASES DA COMPUTAÇÃO PARA DADOS

Para compreender a necessidade e a eficiência das estruturas de dados probabilísticas, é fundamental estabelecer as bases de como os sistemas computacionais lidam com o armazenamento e a recuperação de grandes volumes de informação. Esta seção explora inicialmente as abordagens tradicionais de indexação e os gargalos de acesso a disco, para em seguida detalhar

o funcionamento matemático das funções de espalhamento (*hash*), que servem de alicerce para soluções de busca otimizadas.

2.1.1 Estruturas de Indexação e o Custo de I/O

Em sistemas de gerenciamento de banco de dados e recuperação de informação, um índice consiste em uma estrutura de dados projetada especificamente para localizar registros de forma eficiente, eliminando a necessidade de varreduras lineares exaustivas. O principal objetivo dessas técnicas de indexação é minimizar o número de acessos à memória secundária (operações de Input/Output ou I/O), uma vez que a leitura e gravação em disco consomem significativamente mais tempo e recursos de CPU do que as operações realizadas diretamente na memória principal.

Na prática, os índices operam como caminhos de acesso otimizados. Eles armazenam chaves de busca associadas a ponteiros que indicam a localização física exata do dado no disco, permitindo que o sistema filtre rapidamente registros irrelevantes. A literatura apresenta diversas abordagens para a implementação de índices, cuja escolha depende da natureza dos dados e do perfil de consulta:

- Índices Baseados em Árvores (*Tree-based*): Organizam as chaves em estruturas hierárquicas, como as B-Trees e B+-Trees (JÜRGENS; LENZ, 1999). Por permitirem o armazenamento de ponteiros em nós intermediários e folhas interconectadas, são altamente eficientes para cenários de buscas ordenadas, consultas por intervalos e atualizações frequentes.
- Índices de Bitmap: Utilizam vetores de bits para representar a presença de valores, permitindo operações lógicas (AND, OR, NOT) em altíssima velocidade (MEDINA; BARRANCO; PONS, 2018). São espacialmente eficientes para dados de baixa cardinalidade em ambientes de leitura intensiva.
- Indexação de Texto e Abordagens Modernas: Para bases textuais, destacam-se os Arquivos Invertidos (*Inverted Files*), que mapeiam termos do vocabulário para listas de identificadores de registros (ZOBEL; MOFFAT; RAMAMOHANARAO, 1998). Mais recentemente, surgiram as Estruturas de Índices Aprendidos (*Learned Indexes*), que propõem o uso de modelos de aprendizado de máquina para prever a localização dos dados, substituindo o roteamento tradicional por inferência matemática (OOSTERHUIS; CULPEPPER; RIJKE, 2018).

Apesar de sua eficiência comprovada na localização de registros existentes, as estruturas de indexação determinísticas apresentam um gargalo de desempenho inerente em cenários de dados em larga escala: a busca por elementos ausentes. Quando uma aplicação consulta uma chave que não existe na base de dados, o sistema é obrigado a percorrer a estrutura completa do

índice (por exemplo, navegando da raiz aos nós folhas de uma B-Tree) apenas para, ao final da operação envolvendo discos, confirmar a inexistência do dado.

É exatamente para mitigar este desperdício de I/O e atuar como uma camada de proteção aos índices tradicionais que as estruturas probabilísticas foram idealizadas, operando na memória principal para abortar consultas inúteis antes mesmo que alcancem o disco físico.

2.1.2 Funções de Hash: Propriedades e Algoritmos

As funções de espalhamento, comumente chamadas de funções *hash*, são algoritmos matemáticos projetados para receber um dado de entrada de tamanho arbitrário (como uma *string*, um arquivo ou um objeto complexo) e mapeá-lo para um valor de saída de tamanho fixo, geralmente um número inteiro. No contexto de estruturas probabilísticas como o Filtro de Bloom, as funções de *hash* atuam como o mecanismo que traduz a chave do elemento em posições específicas (índices) dentro de um vetor de bits.

Para ilustrar esse mapeamento na prática, considere um Filtro de Bloom operando com um vetor de $M = 100$ bits e uma função de *hash* $h(x)$. Ao inserir a chave "usuario_123", o algoritmo *hash* pode processar a string e gerar um valor numérico inteiro elevado, como 8.459. Para adequar esse número ao tamanho restrito do vetor, aplica-se a operação aritmética de módulo (resto da divisão), calculando 8.459(mod100). O resultado dessa operação é 59, indicando que a função mapeou a chave para o índice 59 do vetor, alterando o bit daquela posição para 1.

Para que o filtro opere com sua máxima eficiência teórica e prática, a função de *hash* escolhida deve obrigatoriamente apresentar três propriedades fundamentais:

- Determinismo: A função deve ser estritamente determinística, ou seja, ao receber a mesma entrada, ela deve sempre gerar exatamente o mesmo valor de saída. Essa propriedade é essencial, pois o Filtro de Bloom depende da capacidade de recalcular a mesma posição de memória tanto no momento da inserção quanto no momento da consulta.

- Uniformidade (*Avalanche Effect*): O algoritmo deve distribuir os valores de saída da maneira mais uniforme e aleatória possível por todo o espaço disponível. Uma boa função de *hash* evita o "agrupamento"(*clustering*), garantindo que chaves muito parecidas (ex: "teste1"e "teste2") gerem *hashes* completamente diferentes. A uniformidade é a principal responsável por minimizar a taxa de colisões, que ocorre quando chaves diferentes mapeiam para o mesmo índice.

- Velocidade de Computação: Como estruturas probabilísticas são frequentemente utilizadas em gargalos de I/O e processamento em tempo real (como roteamento de redes ou bancos de dados em larga escala), o cálculo do *hash* deve ser extremamente rápido e consumir poucos ciclos de CPU.

Um erro de projeto comum na implementação inicial de estruturas baseadas em *hash* é a utilização de algoritmos criptográficos, como MD5, SHA-1 ou SHA-256 ((PATGIRI; NAYAK; BORGOHAIN, 2019); (NAYAK; PATGIRI; BORAH, 2020); (PATGIRI, 2021)). Funções criptográficas são projetadas sob rígidos requisitos de segurança, com o objetivo de dificultar ataques de colisão maliciosos e engenharia reversa. Esse rigor matemático torna o cálculo dessas funções computacionalmente pesado e lento. Para o Filtro de Bloom tradicional, a resistência a ataques é raramente um requisito, sendo a velocidade e a uniformidade muito mais prioritárias. O uso de algoritmos criptográficos introduz um *overhead* (sobrecarga) de processamento injustificável, que anula o ganho de desempenho que a estrutura propõe.

Portanto, o estado da arte recomenda fortemente o uso exclusivo de famílias de *hashes* não-criptográficos. Nesse cenário, destacam-se dois algoritmos modernos amplamente adotados pela indústria: o CityHash e o MurmurHash (PATGIRI; NAYAK; MUPPALANENI, 2023).

O CityHash, desenvolvido e aberto pela Google, é uma família de funções altamente otimizada para processadores modernos. Seu diferencial é apresentar uma velocidade excepcional ao processar strings, explorando instruções específicas do hardware para realizar leituras e misturas de grandes blocos de *bytes* em poucos ciclos de *clock*.

Apesar da alta velocidade do CityHash, o MurmurHash (criado por Austin Appleby) consolidou-se como a escolha principal e o padrão-ouro para implementações de Filtros de Bloom (AAPPLEBY, 2026). O nome do algoritmo é um acrônimo de suas duas operações matemáticas centrais: multiplicação (*MUltiply*) e rotação de bits (*Rotate*). A grande vantagem arquitetônica do MurmurHash para filtros probabilísticos reside no uso do parâmetro de "semente"(*seed*). Em vez de o desenvolvedor programar múltiplas funções de *hash* diferentes, ele pode invocar o mesmo algoritmo MurmurHash repetidas vezes, alterando apenas a semente de inicialização, para gerar saídas estatisticamente independentes. Aliado a isso, testes empíricos demonstram que a versão MurmurHash2 de 32 bits atinge um equilíbrio perfeito: é extremamente veloz e não desperdiça processamento operando em 64 bits, visto que 32 bits são suficientes para a etapa final de endereçamento do vetor após o cálculo de módulo.

2.2 ESTRUTURAS DE DADOS PROBABILÍSTICAS

Embora as funções de espalhamento (*hash*) e os índices tradicionais ofereçam mecanismos rápidos de busca, a exigência de armazenar as chaves originais para cada registro impõe um limite físico à escalabilidade do sistema. Em cenários de Big Data, onde o volume de elementos distintos pode alcançar a casa dos bilhões, a manutenção de estruturas determinísticas exatas — como HashSets ou árvores de busca — exaure rapidamente a capacidade da memória RAM. Quando isso ocorre, o sistema operacional é forçado a recorrer à paginação em disco rígido (*swap*), o que introduz latência mecânica e degrada severamente o desempenho global das consultas.

Para contornar esse gargalo, a Ciência da Computação consolidou o uso das estruturas de dados probabilísticas. Diferentemente das abordagens determinísticas — que garantem precisão absoluta ao custo de um alto consumo espacial —, as estruturas probabilísticas operam sob um paradigma de perdas aceitáveis (*lossy*). Elas abrem mão de reter os dados originais na íntegra, utilizando vetores compactos de bits para construir assinaturas representativas dos conjuntos. Essa mudança de arquitetura permite representar bases de dados colossais utilizando apenas uma pequena fração da memória original, garantindo que as operações de validação continuem ocorrendo inteiramente na memória principal.

O custo inerente a essa compressão extrema é a introdução de uma margem controlável de erro estatístico nas consultas. Diante deste contexto, esta seção dedica-se a detalhar os conceitos fundamentais que regem o compromisso (*trade-off*) entre a economia de espaço e a precisão das respostas. A seguir, apresentam-se as métricas de avaliação baseadas na matriz de confusão e discutem-se as principais arquiteturas probabilísticas que compõem o estado da arte para diferentes domínios de aplicação.

2.2.1 Conceito e Trade-offs: Espaço vs. Precisão

Estruturas de dados tradicionais, embora determinísticas, apresentam gargalos de memória em cenários de Big Data, pois armazenam a totalidade dos elementos. Em contrapartida, as estruturas de dados probabilísticas atuam comprimindo a representação dos dados. O principal compromisso (*trade-off*) dessa abordagem reside na troca da precisão absoluta pela eficiência de espaço. Em um Filtro de Bloom, por exemplo, é possível economizar gigabytes de memória ao se tolerar uma taxa controlada de Falsos Positivos (frequentemente na casa de 1%), cujo conceito será detalhado na subseção Seção 2.2.2. Destaca-se que, apesar dessa incerteza afirmativa, a estrutura garante a ausência de Falsos Negativos, servindo como uma 'camada de triagem' rigorosa: se o filtro afirma que um elemento não existe, a busca no banco de dados principal pode ser completamente evitada, economizando recursos de I/O.

2.2.2 Avaliação de Precisão: Matriz de Confusão e Métricas

A avaliação de desempenho das estruturas probabilísticas é frequentemente realizada através de um *framework* estatístico conhecido como matriz de confusão, que categoriza os resultados das classificações de pertinência em quatro tipos distintos. Essa taxonomia permite quantificar o comportamento da estrutura frente às consultas no banco de dados:

- **Verdadeiro Positivo (VP):** Ocorre quando um item está, de fato, no conjunto de dados e o filtro responde corretamente que ele está presente.
- **Verdadeiro Negativo (VN):** Ocorre quando um item não está no conjunto e o filtro responde corretamente confirmando a sua ausência. Uma propriedade vital do Filtro de

Bloom padrão é que, se a resposta for negativa, há certeza absoluta de que o dado não existe.

- **Falso Positivo (FP):** Ocorre quando a estrutura classifica incorretamente que um valor consultado faz parte do conjunto, quando na verdade ele não existe na base de dados principal
- **Falso Negativo (FN):** Consiste em classificar um valor existente como inexistente. Embora os Filtros de Bloom tradicionais sejam matematicamente incapazes de produzir falsos negativos (FN = 0), variantes mais complexas que utilizam a "reciclagem"de memória ou permitem a deleção de itens podem introduzir esse erro.

A partir dessa matriz, extraem-se as métricas de avaliação do algoritmo. Na prática computacional com estruturas probabilísticas clássicas, a métrica primordial para atestar a viabilidade do filtro é a **Taxa de Falsos Positivos** (FPR - *False Positive Rate*). A FPR quantifica a proporção de consultas de itens inexistentes que falharam na triagem e resultaram em um alerta incorreto, sendo calculada pela seguinte equação:

$$\text{FPR} = \frac{FP}{FP + VN} \tag{2.1}$$

Em um cenário de sistema de larga escala, o custo computacional de um falso positivo resulta em processamento desperdiçado, como uma busca desnecessária e custosa no disco magnético. Por outro lado, um sistema com alta taxa de verdadeiros negativos e baixa FPR atua como uma camada de triagem de máxima eficiência, bloqueando o acesso ao gargalo de I/O e economizando recursos imediatamente.

2.2.3 Outras Estruturas do Estado da Arte

As estruturas de dados probabilísticas, frequentemente denominadas *sketches*, são fundamentais em áreas como sistemas distribuídos, roteamento, cache e sumarização de dados. Elas são projetadas para fornecer estimativas precisas sobre grandes fluxos de informações, onde a implementação de soluções exatas exigiria quantidades proibitivas de memória.

A escolha da estrutura probabilística a ser implementada depende estritamente do domínio da aplicação e da tolerância ao erro permitida pelo sistema, havendo sempre um compromisso (*trade-off*) entre o uso de recursos computacionais e a precisão. Além do Filtro de Bloom, a literatura contemporânea, como (PATGIRI; NAYAK; MUPPALANENI, 2023), destaca outras abordagens para finalidades distintas:

- *Count-Min Sketch*: Uma estrutura que permite funcionalidades mais amplas do que a pertinência binária, sendo focada na estimativa de frequência de itens (*flow size*) em fluxos contínuos de dados. Trata-se de uma alternativa robusta para consultas de multiplicidade.

- *HyperLogLog*: Um algoritmo projetado para a estimativa de cardinalidade, ou seja, a contagem de elementos distintos em um conjunto. Esta estrutura opera com alta precisão utilizando apenas uma fração da memória que seria necessária para métodos de contagem exatos.

Tabela 1 – Comparação entre estruturas de dados probabilísticas

Estrutura	**Objetivo Principal**	**Trade-off Comum**
Filtro de Bloom	Pertinência de conjunto	Memória vs. taxa de falsos positivos.
Count-Min Sketch	Estimativa de frequência	Memória vs. erro de superestimativa.
HyperLogLog	Cardinalidade de conjunto	Memória vs. erro de estimativa.

Fonte: Elaborado pelo autor.

3 O FILTRO DE BLOOM

Conforme compilado na obra *Bloom Filter: A Data Structure for Computer Networking, Big Data, Cloud Computing, Internet of Things* (PATGIRI; NAYAK; MUPPALANENI, 2023), o Filtro de Bloom é, em sua essência, uma estrutura de dados probabilística utilizada para filtragem de pertinência (*membership filtering*). Sua principal premissa é atestar, com absoluta certeza, a inexistência de uma chave em um conjunto de dados. Sob a ótica arquitetônica, a estrutura consiste em um vetor binário, inicialmente preenchido com zeros, operado por um conjunto de k funções de espalhamento (*hash*) independentes, as quais geram posições específicas no vetor para cada chave inserida.

O objetivo primordial desta estrutura é interceptar e reduzir o número de consultas no banco de dados referentes a valores que não existem em seus registros. Essa filtragem prévia resulta em uma diminuição significativa do custo de processamento e do tempo de resposta, sobretudo porque buscas mal-sucedidas no armazenamento secundário são altamente custosas e tendem a atingir a complexidade máxima de varredura dos algoritmos de busca. Adicionalmente, por utilizar uma representação compacta baseada puramente em bits, o vetor pode ser alocado integralmente na memória principal (RAM) da máquina, acelerando ainda mais o processo de verificação.

Diante da relevância desta estrutura, este capítulo detalha os mecanismos lógicos que compõem o Filtro de Bloom. Nas subseções a seguir, serão examinados o funcionamento das operações de inserção e consulta, a fundamentação matemática que rege a probabilidade de falsos positivos e a arquitetura das funções de *hash* que o integram.

3.1 FUNCIONAMENTO: INSERÇÃO E CONSULTA

Para compreender a dinâmica operacional do Filtro de Bloom, é necessário primeiramente contextualizar o fluxo de processamento de dados sob uma perspectiva prática de engenharia de software. Considere um cenário típico de sistemas de grande escala onde um conjunto massivo de dados encontra-se armazenado de forma persistente em um arquivo em disco ou em um banco de dados secundário. Antes que qualquer verificação de pertinência possa ser executada com eficiência de tempo constante, o sistema precisa instanciar e inicializar a estrutura probabilística na memória principal (RAM), configurando um vetor de bits de tamanho m totalmente zerado e mapeando as k funções de espalhamento que orquestrarão o algoritmo.

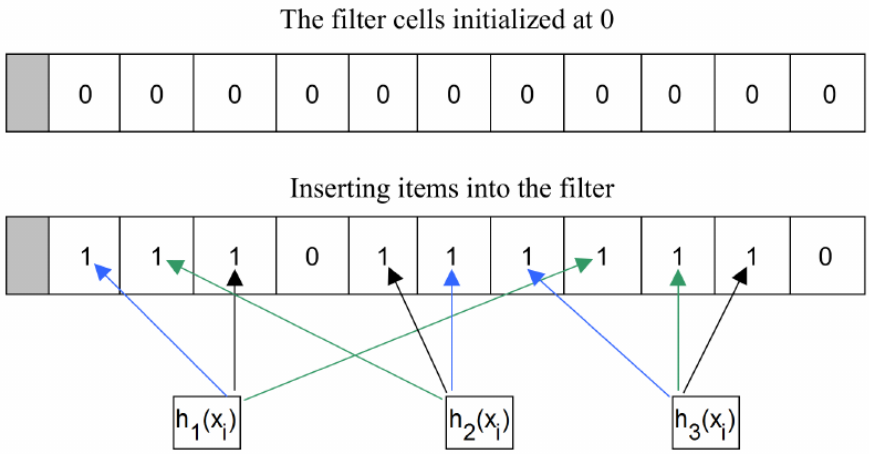
A partir dessa estrutura inicial estéril, o ciclo de vida do filtro inicia-se obrigatoriamente através de uma fase de população ou construção (*build phase*). O sistema realiza a leitura sequencial do arquivo de dados e, para cada registro ou chave identificada, executa a operação de inserção. Esse processo consiste em submeter a chave às k funções *hash* independentes, calcu-

lando os índices correspondentes dentro do arranjo unidimensional e alterando o estado desses bits específicos para o valor lógico 1. Uma vez que todo o conjunto de dados original contido no arquivo foi processado e indexado, o vetor de bits passa a atuar como uma assinatura compacta e estável daquela base de dados.

Com a estrutura devidamente instanciada e populada a partir do arquivo, o Filtro de Bloom assume o seu papel operacional pleno no ecossistema de software. A partir deste ponto, o sistema pode gerenciar com eficiência as duas ações fundamentais que definem o comportamento da estrutura: a inclusão contínua de novos elementos em tempo de execução e a pesquisa de dados (consulta) para mitigar o gargalo de I/O em buscas mal-sucedidas. A seguir, detalham-se os mecanismos algorítmicos e lógicos que governam cada uma dessas operações essenciais.

A operação de inserção (representada na figura Figura 1), também denominada preparo ou criação, consiste em um procedimento estruturado em etapas simples. Primeiramente, o novo elemento (ou chave) a ser adicionado é submetido a k funções de *hash*. Essas funções retornam k valores numéricos distintos, que são mapeados para posições específicas do vetor de bits por meio de uma operação de módulo ($\mathrm{mod} m$, onde m é o tamanho do vetor). Após esse mapeamento, os bits correspondentes a essas k posições são ativados (isto é, alterados para o valor 1).

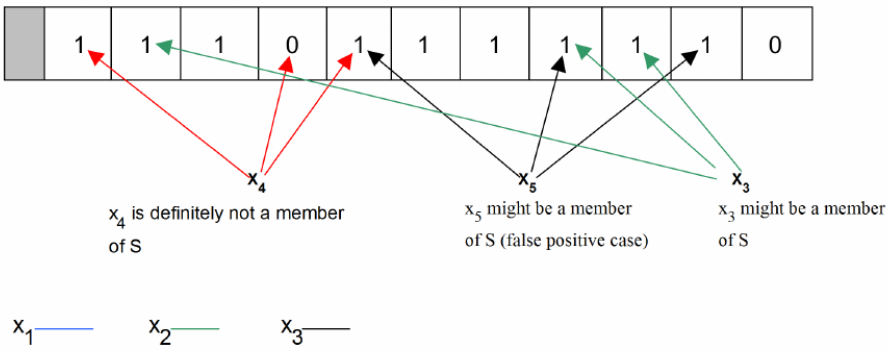
Figura 1 – Representação visual dos processos de inserção no filtro.



Fonte: Retirado de Abdennebi e Kaya (2021).

A operação de consulta, representada na figura Figura 2, por sua vez, tem como principal objetivo validar a inexistência de um determinado elemento no conjunto. Para realizar essa verificação, a chave procurada é processada pelas mesmas k funções de *hash*, o que resulta em k posições no vetor. Por fim, verifica-se o estado dos bits nessas respectivas posições: se pelo menos um dos bits estiver desligado (com valor 0), pode-se afirmar com certeza absoluta que a chave não pertence ao conjunto.

Figura 2 – Representação visual dos processos de consulta no filtro.



Fonte: Retirado de Abdennebi e Kaya (2021).

3.2 ANÁLISE MATEMÁTICA: PROBABILIDADE DE FALSOS POSITIVOS (FPR)

A principal característica do Filtro de Bloom é a ocorrência de falsos positivos, cuja probabilidade está diretamente ligada a três parâmetros fundamentais da estrutura: o tamanho do vetor de bits (m), o número de elementos inseridos no conjunto (n) e a quantidade de funções de espalhamento independentes utilizadas (k).

Para compreender a taxa de falsos positivos (do inglês, *False Positive Rate* - FPR), é necessário analisar a probabilidade de um bit específico do vetor permanecer com o valor 0 após a inserção de n elementos. Assumindo que as funções de *hash* distribuem os valores de forma perfeitamente uniforme, a probabilidade de um determinado bit não ser alterado para 1 por uma única função de *hash* durante uma inserção é de $1-\frac{1}{m}$ (PATGIRI; NAYAK; MUPPALANENI, 2023).

Como o filtro utiliza k funções de *hash*, a probabilidade de o bit permanecer 0 após uma inserção completa é $(1-\frac{1}{m})^k$. Após a inserção de n elementos, essa probabilidade passa a ser:

$$(1-\tfrac{1}{m})^{kn}$$

Utilizando propriedades de limites fundamentais, essa expressão pode ser aproximada para $e^{-kn/m}$, utilizando a constante de Euler (e). Consequentemente, a probabilidade de um bit qualquer possuir o valor 1 após todas as inserções é o evento complementar: $1-e^{-kn/m}$.

Um falso positivo ocorre durante uma consulta quando um elemento que não foi inserido tem todas as suas k posições mapeadas para bits que já estão com o valor 1 (devido a colisões de elementos anteriores). Como as funções de *hash* são independentes, a probabilidade P de isso acontecer é a probabilidade de um bit ser 1 elevada à potência k:

$$P = (1-e^{-kn/m})^k$$

3.2.1 A Otimização: A Relação Ideal entre m, n e k

A equação de probabilidade do falso positivo demonstra um comportamento de *trade-off* no momento de parametrizar o filtro. Se o valor de k (número de *hashes*) for muito baixo, a chance de colisão aumenta porque há poucos bits sendo verificados. Por outro lado, se k for muito alto, o vetor de bits será preenchido rapidamente (muitos bits alterados para 1 a cada inserção), o que também aumenta drasticamente a taxa de falsos positivos.

Para minimizar a probabilidade P para um vetor de tamanho m e um número de elementos n pré-estabelecidos, calcula-se a derivada da equação em relação a k. Ao igualar a derivada a zero, chega-se à fórmula do número ótimo de funções de *hash* (PATGIRI; NAYAK; MUPPALANENI, 2023):

$$k = \frac{m}{n} \ln(2)$$

Essa relação, que utiliza logaritmo natural, comprova que o número ideal de funções de espalhamento (k) cresce linearmente em relação à proporção de bits disponíveis por elemento inserido (m/n). Na prática, quando o filtro opera com esse k ótimo, a probabilidade de falso positivo decai exponencialmente conforme se aumenta a proporção de bits alocados por elemento.

3.3 LIMITAÇÕES DO FILTRO ORIGINAL

Entretanto, o BF tem seus contras. O principal, já citado, são os falsos positivos, que são os casos onde duas ou mais chaves podem gerar em conjunto as k posições que uma chave X, que não existe, geraria. Esses casos são chamados de colisões, e por isso que o filtro não pode garantir a existência de um valor, sendo necessário pesquisar ele no banco de dados se não tiver nenhum valor 0 dos bits de suas k posições.

Outra questão é o tamanho. Por ser um vetor ele tem tamanho fixo, então, conforme o conjunto de dados cresce e o espaço não, ele vai se tornando obsoleto pelos casos cada vez mais recorrentes de colisões. Para isso existem os cálculos citados na seção Seção 3.2 que definem tamanho e quantidade de funções de *hash* ideais para um número n de dados para alcançar uma precisão P boa.

Adicionalmente, não se pode deletar valores. Como 2 ou mais chaves diferentes podem usar a mesma posição é impossível saber se o bit de valor 1 de uma posição Y é exclusivo de uma chave X ou não sem refazer o filtro inteiro novamente, logo não podemos zerar ele.

Outro problema é a escala, quanto mais funções de *hash*, e mais demoradas elas forem, mais tempo e processamento custará para pesquisar, mas principalmente para gerar o filtro. Isso tem o custo computacional medido pelo cálculo de Complexidade vezes k vezes n citado na seção Seção 3.2.

3.4 VARIANTES DO FILTRO DE BLOOM

Apesar da sua notável eficiência teórica na redução de operações de I/O[1] e economia de espaço, o Filtro de Bloom Padrão (Standard Bloom Filter - SBF) (BLOOM, 1970) apresenta limitações arquitetônicas e funcionais quando submetido a cenários contemporâneos de Big Data e redes dinâmicas de alta velocidade. As restrições inerentes à estrutura original — nomeadamente a impossibilidade matemática de remoção de itens sem comprometer a integridade do conjunto e os gargalos de desempenho causados por acessos aleatórios à memória (*cache misses*) — motivaram a comunidade científica a desenvolver diversas extensões ao longo das últimas décadas.

Nesta seção, são exploradas três variantes fundamentais que procuram mitigar estes desafios existentes (ABDENNEBI; KAYA, 2021), ordenadas pela sua complexidade estrutural: o Filtro de Bloom com Contagem, que resolve o problema da mutabilidade dos dados; o Filtro de Bloom Particionado, focado na otimização do hardware e paralelismo; e o Filtro de Bloom r-Dimensional, que representa uma reestruturação topológica profunda para maximizar a velocidade, precisão e escalabilidade em sistemas críticos.

3.4.1 Filtro de Bloom com Contagem (*Counting Bloom Filter)*

O Filtro de Bloom Padrão apresenta uma restrição severa em ambientes de dados altamente dinâmicos: a impossibilidade matemática de remover elementos do conjunto. Devido à sobreposição de chaves em índices idênticos (decorrente de colisões naturais das funções de *hash*), reverter um valor de 1 para 0 ao deletar um item poderia inadvertidamente apagar o registro de outros elementos válidos que compartilham aquela posição. Essa ação introduziria falsos negativos na estrutura, violando sua propriedade mais fundamental. Para sanar essa limitação, o Filtro de Bloom com Contagem (*Counting Bloom Filter* - CBF) foi desenvolvido como uma das expansões arquitetônicas mais críticas da estrutura original (TARKOMA; ROTHENBERG; LAGERSPETZ, 2012).

A alteração central introduzida pelo CBF consiste na substituição do vetor de bits simples por um vetor de pequenos contadores numéricos inteiros. A dinâmica de inserção é adaptada: quando um novo elemento é mapeado pelas k funções de espalhamento, em vez de simplesmente alterar o estado lógico da memória, o sistema incrementa o valor numérico dos contadores naquelas posições específicas em uma unidade. A remoção de elementos torna-se viável de forma simétrica; calculam-se as k posições da chave a ser excluída e os contadores correspondentes são decrementados. A etapa de consulta verifica se todos os contadores mapeados possuem valor estritamente maior que zero; a presença de qualquer contador igual a zero é

[1] Em Arquitetura de Computadores, o gargalo de I/O (Input/Output) refere-se à disparidade de velocidade entre a CPU e os dispositivos de armazenamento secundário (como HDDs e SSDs). Devido à alta latência mecânica ou de barramento físico na leitura de discos, operações que exigem buscas constantes fora da memória principal (RAM) degradam severamente o desempenho do sistema.

garantia absoluta de que o elemento não está no conjunto.

O compromisso (*trade-off*) exigido para habilitar a deleção incide substancialmente sobre a eficiência espacial. A literatura convenciona que contadores de 4 bits são adequados para a maioria das aplicações, o que faz com que o CBF consuma, nativamente, quatro vezes mais memória do que o filtro padrão para suportar o mesmo volume de dados. Além desse custo espacial, a estrutura exige tratamento contra o transbordamento aritmético (*overflow*). Considerando um contador de 4 bits (cujo valor máximo é 15), se as colisões elevarem o contador além de sua capacidade, a abordagem padrão consiste em mantê-lo permanentemente travado em seu valor máximo. O impacto negativo dessa mitigação não ocorre durante a inserção, mas sim no momento da exclusão. Se for realizada a exclusão de um item que utiliza esse contador travado, o valor não será decrementado, deixando um "registro fantasma". Como o contador nunca mais retornará a zero, mesmo após a exclusão dos elementos que ali colidiram, o filtro perde o rastreamento exato das deleções naquelas posições, o que eleva a probabilidade de falsos positivos em consultas futuras.

Para mitigar o alto consumo espacial e os desafios de precisão do CBF original, extensões modernas foram desenvolvidas na literatura recente. Abordagens como o *d-left Counting Bloom Filter* utilizam *hashing* assimétrico e impressões digitais (*fingerprints*) para reduzir a exigência de espaço pela metade. De forma paralela, variantes como o *Variable-Increment CBF* (VI-CBF) alteram o protocolo de incremento unitário para incrementos variáveis, visando diminuir as colisões locais e melhorar a exatidão em coleções de extrema escala. Em suma, o CBF e suas ramificações são indispensáveis em arquiteturas de *Big Data*, onde a mutabilidade dos dados exige que a estrutura de filtragem seja capaz de remover e "esquecer"informações obsoletas.

3.4.2 Filtro de Bloom Particionado (*Partitioned Bloom Filter)*

O Filtro de Bloom Particionado (TARKOMA; ROTHENBERG; LAGERSPETZ, 2012) surge como uma variante arquitetônica desenhada para otimizar o acesso à memória e viabilizar a execução paralela em nível de hardware. Diferente da abordagem tradicional, na qual todas as k funções de espalhamento operam e distribuem bits ao longo de todo o espaço global do filtro, a variante particionada divide a estrutura em blocos independentes. Matematicamente, o vetor original de tamanho m é fragmentado em k partições disjuntas, resultando em sub-vetores de tamanho m/k.

Nesse modelo restrito, o mapeamento torna-se estritamente localizado: cada função de *hash* hi recebe permissão para alterar ou consultar apenas os bits contidos na sua respectiva partição i. Durante a inserção de um novo elemento, o sistema calcula uma posição para cada uma das k partições e altera os respectivos bits para 1. O processo de consulta segue a mesma lógica geométrica: o algoritmo verifica um bit específico dentro de cada partição; caso qualquer um desses bits possua valor 0, confirma-se com absoluta certeza matemática que o elemento

não pertence ao conjunto.

Do ponto de vista do desempenho assintótico teórico[2], o filtro particionado é equivalente ao Filtro de Bloom padrão. Contudo, há um leve *trade-off* prático em relação à precisão. Como o espaço de endereçamento de cada função de *hash* é reduzido ao tamanho da partição (m/k), a densidade de bits com valor 1 (fator de preenchimento) tende a crescer mais rapidamente dentro desses blocos restritos. Esse preenchimento acelerado eleva ligeiramente a probabilidade de colisões locais, resultando em uma taxa de falsos positivos marginalmente superior à do filtro operando em espaço global.

Apesar desse pequeno sacrifício na precisão, a adoção do particionamento é amplamente justificada por suas vantagens mecânicas. A principal motivação para o uso desta variante reside na eliminação de conflitos de concorrência (*locks* de memória). Como as funções de *hash* operam em intervalos de memória fisicamente separados, múltiplas *threads* podem realizar inserções e consultas em paralelo sem disputar o mesmo segmento. Além disso, essa organização espacial fundamenta outras otimizações avançadas, como o *One-Hashing Bloom Filter*, que reduz drasticamente a carga computacional ao utilizar apenas uma função de *hash* principal, combinada com operações aritméticas de módulo, para distribuir o mapeamento ao longo das partições. Em síntese, o modelo particionado troca uma fração mínima de exatidão por um ganho massivo em velocidade de processamento e paralelismo.

3.4.3 Filtro de Bloom r-Dimensional (rDBF)

Seguindo a progressão de complexidade das variantes estruturais, o Filtro de Bloom r-Dimensional (rDBF) (PATGIRI, 2021) apresenta-se como uma reestruturação topológica profunda do modelo original. Em vez de operar sobre um vetor linear unidimensional de bits, o rDBF projeta e mapeia os elementos em arranjos matemáticos multidimensionais (podendo operar em 2D, 3D, 4D, entre outros). Uma exigência matemática rigorosa e definidora desta arquitetura é que os tamanhos das dimensões do arranjo (os eixos X,Y,Z, etc.) sejam formados exclusivamente por números primos. Essa propriedade é fundamental para garantir que as operações aritméticas de módulo consigam espalhar os mapeamentos de maneira perfeitamente uniforme por todo o espaço multidimensional, mitigando drasticamente os agrupamentos (*clustering*) e as colisões estruturais que ocorreriam em matrizes de dimensões pares ou compostas.

O maior diferencial computacional do rDBF é a quebra do paradigma em relação ao custo de *hashing*. Em Filtros de Bloom clássicos e suas variantes imediatas, o custo de CPU escala linearmente com o número de funções de espalhamento (k). O rDBF torna-se independente dos parâmetros k e do tamanho da entrada (n) ao evitar a execução repetida de múltiplos algoritmos de *hash*. A estrutura utiliza uma única chamada de uma função de espalhamento de

[2] Na Ciência da Computação, o desempenho assintótico refere-se ao comportamento do tempo de execução ou uso de memória de um algoritmo à medida que o volume de dados de entrada (n) cresce tendendo ao infinito, sendo formalizado pela Notação Big-O. No caso do Filtro de Bloom, este comportamento teórico é constante, representado por O(k)

altíssimo rendimento (frequentemente o MurmurHash) e deriva todas as coordenadas espaciais necessárias estritamente através de operações sucessivas de módulo sobre o valor gerado. A manipulação no hardware é executada diretamente sobre matrizes de inteiros do tipo *unsigned long*, explorando operações *bitwise* primitivas (como deslocamentos lógicos e portas OR) que são resolvidas em pouquíssimos ciclos de *clock* pelo processador.

Em termos de eficiência espacial e funcional, o rDBF alcança um limite ótimo ao garantir que cada item inserido exija, no máximo, o custo equivalente a um único bit na estrutura global. De forma surpreendente para sua densidade, essa variante multidimensional também suporta a operação de deleção de itens sem a contrapartida de introduzir falsos negativos, superando o principal dilema do filtro original. Devido a essa conjunção ímpar de acurácia milimétrica, velocidade de roteamento e extrema economia de RAM, o rDBF consolidou-se como uma solução canônica para arquiteturas de rede de próxima geração, sendo amplamente adotado em roteadores baseados em Named Data Networking (NDN) para gerenciar componentes de busca em tempo real, como a Content Store (CS) e a Pending Interest Table (PIT).

3.5 APLICAÇÕES MODERNAS E FERRAMENTAS

A transição das estruturas probabilísticas do campo estritamente teórico para a engenharia de software aplicada reflete a necessidade premente de resolver problemas reais de escalabilidade. Embora as secções anteriores tenham detalhado os fundamentos matemáticos e as adaptações arquitetônicas do Filtro de Bloom, a verdadeira força desta estrutura reside na sua adoção massiva por parte da indústria tecnológica contemporânea. Este vetor de otimização provou que tolerar uma margem controlada de incerteza é um compromisso altamente rentável quando se trata de poupar recursos críticos.

Nesta seção, procura-se demonstrar que o Filtro de Bloom e as suas variantes ultrapassaram o estatuto de curiosidades académicas para se tornarem pilares essenciais em sistemas de grande escala. Para tal, serão explorados, primeiramente, os casos de uso prático em infraestruturas de corporações globais — onde a estrutura mitiga diariamente gargalos de latência e de I/O — e, subsequentemente, analisar-se-ão as principais bibliotecas e implementações técnicas que disponibilizam este poder computacional aos programadores, estabelecendo assim a base material e tecnológica que fundamentará a metodologia prática deste trabalho.

3.5.1 Casos de Uso na Indústria (Sistemas de Larga Escala)

A adoção do Filtro de Bloom pela indústria de tecnologia contemporânea demonstra sua eficácia prática em mitigar gargalos de latência (PATGIRI; NAYAK; BORGOHAIN, 2019). A sua principal função em arquiteturas corporativas é atuar como uma camada de triagem de altíssima velocidade, evitando que operações computacionalmente custosas — como varreduras

em discos magnéticos ou requisições de rede intercontinentais — sejam executadas desnecessariamente em cenários onde o volume de dados atinge a escala de *gigabytes* ou *terabytes*.

O uso industrial mais emblemático da estrutura ocorre nos sistemas modernos de banco de dados e armazenamento de *Big Data*. O sistema Google BigTable (TARKOMA; ROTHENBERG; LAGERSPETZ, 2012), por exemplo, emprega Filtros de Bloom para gerenciar os seus arquivos de dados estruturados e reduzir drasticamente as requisições ao disco rígido (HDD). Antes de iniciar a leitura física para localizar uma linha ou coluna específica, o sistema consulta o filtro alocado na memória RAM; se o retorno for negativo, o BigTable aborta a operação de I/O imediatamente. A mesma lógica arquitetônica é replicada em bancos de dados NoSQL de altíssimo rendimento, como o Apache Cassandra e o HBase, que utilizam variantes do filtro para mapear suas tabelas imutáveis em disco (*SSTables*), permitindo gerenciar *petabytes* de dados e garantindo a escalabilidade horizontal.

Além dos bancos de dados tradicionais orientados a disco, a evolução das arquiteturas de microsserviços e sistemas em tempo real consolidou a integração de estruturas probabilísticas diretamente em motores de armazenamento em memória (*in-memory data stores*). Ferramentas amplamente adotadas no mercado, como o Redis (REDIS, 2022) e o ecossistema de código aberto Valkey — este último gerenciado em nuvem através do serviço Google Cloud Memorystore —, disponibilizam o Filtro de Bloom nativamente como um tipo de dado primitivo de alta performance. Nesses ambientes, a estrutura assume um papel crítico no desenvolvimento de sistemas de mensageria, plataformas de *streaming* e portais de *e-commerce*. O principal caso de uso nestes cenários é a desduplicação massiva de eventos e anúncios: o filtro permite determinar instantaneamente se um determinado usuário já visualizou uma peça publicitária, recebeu um *e-mail* promocional ou acionou uma notificação no aplicativo. Ao interceptar essas validações diretamente na memória RAM por meio de operações probabilísticas rápidas, previne-se a saturação das camadas de persistência e acelera-se o tempo de resposta global do ecossistema.

Além do armazenamento persistente e em memória, a segurança e a infraestrutura de navegação na Web dependem profundamente de estruturas probabilísticas. Um caso de uso clássico é o de navegadores (como o Google Chrome em suas implementações de *Safe Browsing*) (GOOGLE, 2026), que utilizam os filtros para verificar de forma instantânea se uma URL acessada pelo usuário pertence a uma base de dados de *phishing* ou *malware*. Em vez de forçar o navegador a fazer o *download* de uma lista gigantesca contendo milhões de URLs maliciosas (o que consumiria excessiva memória no cliente) ou realizar uma consulta de rede a cada clique do usuário (o que geraria latência insuportável), o navegador baixa apenas o Filtro de Bloom compacto. O custo ocasional de um falso positivo neste cenário resulta apenas em uma verificação de rede secundária para confirmar a ameaça, o que é um *trade-off* altamente vantajoso que protege inclusive a privacidade de navegação do usuário.

Por fim, no tráfego de redes de computadores e controle de *spam* (BRODER; MITZENMACHER, 2004), o Filtro de Bloom opera na triagem de pacotes em milissegundos. Provedores

de correio eletrônico utilizam a estrutura para manter registros massivos de remetentes ou domínios bloqueados (*blocklists*), descartando conexões indesejadas na borda antes mesmo de processar o corpo das mensagens. Paralelamente, em redes de grande escala (NAYAK; PATGIRI; BORAH, 2020), ferramentas como o Squid Web Proxy e diversas *Content Delivery Networks* (CDNs) utilizam a estrutura para criar "resumos de cache"(*cache digests*). Em vez de os servidores trocarem listas exaustivas dos arquivos que possuem armazenados, eles trocam Filtros de Bloom compactos. Isso permite que um roteador ou servidor descubra instantaneamente se um nó vizinho possui o recurso solicitado, otimizando o roteamento e mitigando requisições redundantes de pacotes.

3.5.2 Bibliotecas e Implementações Disponíveis

Devido à necessidade rigorosa de otimização de espaço e à constante manipulação de posições de memória em baixo nível através de operações bitwise, a linguagem C consolidou-se como o padrão-ouro para a implementação de Filtros de Bloom e suas variantes. Uma das bibliotecas de código aberto mais consagradas e utilizadas como *benchmark* de desempenho na indústria é a libbloom (JVIRKKI, 2022). O grande diferencial desta biblioteca é a sua abstração matemática: a interface de inicialização $bloom_init2$ não exige que o desenvolvedor calcule manualmente o tamanho do vetor ou o número de funções de *hash*. O programador fornece apenas o número esperado de elementos a serem inseridos $porexemplo, 1.000.000$ e a probabilidade de erro tolerável $como 0,01$. Com base nessas entradas, a biblioteca calcula dinamicamente o número ideal de bits e de funções de espalhamento em tempo de execução, alocando a memória necessária. As operações subsequentes de inserção e consulta $bloom_addebloom_check$ recebem ponteiros genéricos de memória $void * buffer$ e seus respectivos comprimentos, processando-os diretamente através do algoritmo não-criptográfico MurmurHash2.

No ambiente de pesquisa acadêmica voltado para estruturas de próxima geração, o ecossistema de código aberto em C continua a expandir os limites da arquitetura clássica. Um exemplo moderno é a biblioteca robustBF (PATGIRI, 2021), desenvolvida para instanciar um filtro bidimensional (2D). Do ponto de vista de implementação, os parâmetros de entrada desta variante exigem que as dimensões da matriz (eixos X e Y) sejam obrigatoriamente números primos, uma restrição técnica para garantir o espalhamento uniforme pelas operações de módulo. Em vez de utilizar vetores de bits tradicionais, a robustBF estrutura sua matriz utilizando variáveis do tipo unsigned long int. Essa escolha de tipagem em C não é acidental; ela permite que a estrutura processe manipulações aritméticas primitivas em pouquíssimos ciclos de *clock* do processador, viabilizando, inclusive, a exclusão de registros por meio da aplicação do operador lógico XOR. Assim como a libbloom, a robustBF fundamenta sua eficiência em uma variação do mesmo algoritmo de espalhamento (MurmurHash H4), comprovando que a união entre a linguagem C e a família MurmurHash constitui o arcabouço técnico ideal para sistemas probabilísticos.

4 PROJETO DE DESENVOLVIMENTO

O objetivo central deste capítulo é delimitar o percurso metodológico, os procedimentos técnicos e o desenho experimental delineados para avaliar, de forma empírica, o desempenho e a viabilidade de diferentes arquiteturas de Filtros de Bloom frente a cenários de processamento massivo de dados. Enquanto a literatura científica consagra as formulações matemáticas abstratas e os limites assintóticos dessas estruturas probabilísticas, faz-se imperativo transpor tais conceitos para o ambiente de execução real. Elementos práticos como a arquitetura do processador, o comportamento da memória cache, a eficiência computacional das funções de espalhamento e a distribuição de entropia nas chaves de entrada introduzem variáveis que impactam diretamente a latência e a taxa de falsos positivos, justificando a necessidade de uma investigação prática controlada.

Para alcançar esse propósito, esta pesquisa caracteriza-se como um estudo de natureza aplicada, com abordagem quantitativa e método eminentemente experimental. A estratégia experimental baseia-se no isolamento rigoroso de variáveis independentes — tais como o tamanho do vetor de bits, a quantidade de funções hash computadas e a tipologia estrutural dos dados de entrada — para observar e mensurar seus efeitos sobre as variáveis dependentes, representadas pelo consumo de memória RAM em megabytes, pela latência de criação do índice (*build time*) e pelo tempo de resposta das consultas (lookup time). Dessa forma, a metodologia não visa apenas implementar algoritmos isolados, mas sim construir um ecossistema comparativo capaz de mapear com exatidão os trade-offs (compromissos entre espaço e precisão) inerentes a cada modelo.

A engenharia da solução proposta está estruturada de modo a simular os gargalos de I/O de sistemas operacionais e de bancos de dados em larga escala. O delineamento da pesquisa prevê a execução de testes estressantes de pertinência, confrontando o Filtro de Bloom Padrão (Standard) e suas variantes evolutivas — incluindo modelos de contagem (Counting), particionados, multidimensionais (r-Dimensional) e uma abordagem híbrida integrada — contra uma linha de base determinística de referência. Através deste arranjo experimental, busca-se fornecer subsídios empíricos para validar em que medida as otimizações estruturais propostas conseguem mitigar colisões e reduzir o consumo de memória sem comprometer a integridade e a velocidade exigidas pelo estado da arte em Big Data.

4.1 ARQUITETURA DA SOLUÇÃO E MODELOS PROPOSTOS

A concepção da arquitetura do ambiente de testes assenta num paradigma estritamente modular. Esta escolha arquitetônica justifica-se pela necessidade de desenvolver, testar e isolar o comportamento de múltiplas estruturas de dados em paralelo, permitindo que as avaliações

de inserção, consulta e deleção ocorram sem interferências cruzadas. O sistema global será composto por um orquestrador central e por um conjunto progressivo de "módulos de armazenamento", que representarão a evolução do Filtro de Bloom, culminando num modelo híbrido proposto por este trabalho.

A esteira de desenvolvimento metodológico seguirá a seguinte progressão de implementações:

1. Filtro de Bloom Padrão (SBF): O módulo fundamental consistirá na alocação de um vetor linear estático de bits. A interação com este vetor não recorrerá a abstrações de alto nível, mas sim a operações *bitwise* puras, servindo como a base de performance intrínseca de tempo e espaço com a qual as variantes subsequentes serão comparadas.

2. Filtro de Bloom de Contagem (CBF): Para superar a limitação matemática da imutabilidade, o segundo módulo substituirá o vetor de bits por um vetor de contadores inteiros (variáveis de tamanho fixo em C). Este modelo introduzirá a funcionalidade de deleção de chaves do *dataset*, permitindo mensurar o custo extra de memória (em bytes) exigido para manter a flexibilidade dos dados sem gerar falsos negativos.

3. Filtro de Bloom Particionado (PBF): O terceiro módulo implementará a divisão física do espaço de endereçamento. O vetor global será fragmentado em blocos matematicamente disjuntos, limitando cada variação da semente do algoritmo *hash* a mapear índices apenas dentro da sua respectiva partição. O objetivo é mensurar empiricamente o impacto deste particionamento na velocidade de I/O em memória e na mitigação de colisões globais.

4. Filtro de Bloom r-Dimensional (rDBF): O quarto módulo trará a quebra do paradigma linear, projetando o filtro sobre matrizes multidimensionais onde as dimensões são estritamente números primos. A implementação focará na economia extrema de processamento, invocando a função de espalhamento apenas uma vez e derivando as demais coordenadas espaciais via operações de módulo.

5. Modelo Probabilístico Híbrido (Proposta Integrada): Como ápice da arquitetura proposta, será desenvolvido um modelo que amalgama as características arquitetônicas das variantes anteriores. Convém destacar que a fusão dessas abordagens (a economia de chamadas *hash* do rDBF, o isolamento de memória do PBF e a mutabilidade via contadores do CBF) impõe desafios lógicos complexos e potenciais conflitos de *trade-off*. Sob a ótica deste trabalho científico, o objetivo primordial desta estrutura não é presumir uma otimização infalível, mas sim investigar empiricamente se a sua sinergia é viável ou se o custo cumulativo de suas restrições degrada a eficiência geral, mapeando de forma rigorosa os limites práticos dessa hibridização frente aos gargalos de *Big Data*.

Para garantir a validade científica das comparações, todos os modelos probabilísticos acima serão confrontados com uma Linha de Base Determinística (*Deterministic Baseline*).

Será utilizada uma estrutura do tipo *HashSet* clássica para armazenar a totalidade dos dados brutos do *dataset*. Tendo em vista que a linguagem C não possui uma implementação nativa de tabelas de espalhamento, e visando mitigar vieses de desempenho decorrentes de uma implementação própria subótima, este trabalho adotará a biblioteca de código aberto `uthash` (TROYDHANSON, 2026). Amplamente consolidada e otimizada por meio de macros na literatura prática, a `uthash` garantirá que a linha de base opere com máxima eficiência determinística (complexidade média de $O(1)$). Este modelo servirá de âncora absoluta: apresentará zero por cento de erros, permitindo contrastar de forma irrefutável a cobrança exponencial de memória RAM contra a economia espacial alcançada pela esteira probabilística desenvolvida em C e alimentada pelo MurmurHash2.

4.2 TECNOLOGIAS E FERRAMENTAS DE DESENVOLVIMENTO

A seleção do ecossistema tecnológico para a implementação dos modelos propostos foi guiada por dois requisitos metodológicos inegociáveis: o controle absoluto sobre a alocação de memória e a capacidade de medir o tempo de execução (latência) sem a interferência de processos em segundo plano. Por conseguinte, a totalidade dos módulos estruturais (tanto os filtros probabilísticos como a linha de base determinística) será codificada utilizando a linguagem C, especificamente sob o padrão ANSI C99.

A escolha da linguagem C justifica-se por três fatores arquitetônicos. Em primeiro lugar, a ausência de um *Garbage Collector* (coletor de lixo automático, presente em linguagens de alto nível como Java ou Python) garante que os cronômetros de teste meçam exclusivamente o tempo de processamento dos algoritmos, evitando picos de latência (*spikes*) causados por limpezas de memória em segundo plano. Em segundo lugar, a manipulação das estruturas probabilísticas exige a alteração cirúrgica de bits individuais, uma tarefa que a linguagem C executa com extrema rapidez através de operações *bitwise* nativas (deslocamentos lógicos e máscaras booleanas) que são traduzidas quase diretamente para a linguagem de máquina do processador. Por fim, a adoção do padrão C99 permite a utilização da biblioteca `<stdint.h>`, garantindo a instanciação de variáveis de tamanho rigorosamente fixo, como o `uint32_t`, algo essencial para provar matematicamente a economia de bytes em relação à estrutura de referência. A compilação do código será efetuada através do GCC (*GNU Compiler Collection*), com as diretivas de otimização máxima de desempenho ativadas (sinalizador `-O3`).

Ademais, para viabilizar os testes de estresse em cenários de processamento massivo de dados e simular a concorrência típica de sistemas distribuídos e bancos de dados, a arquitetura de software explorará o paralelismo em nível de *thread*. Para tanto, far-se-á uso da biblioteca nativa POSIX Threads (`<pthread.h>`). A introdução de multiprocessamento permitirá que as operações de consulta (*lookup*) e inserção sejam distribuídas uniformemente entre os núcleos físicos do processador, minimizando os tempos ociosos do CPU e garantindo que as métricas de latência reflitam o comportamento das estruturas sob alta concorrência. O controle de exclusão

mútua via travas (*mutexes*), necessário nas variantes mutáveis (como o CBF e o modelo híbrido) para evitar condições de corrida (*race conditions*), também será monitorado, enriquecendo a análise de escalabilidade das soluções.

No que diz respeito ao algoritmo de espalhamento, o projeto integrará o MurmurHash2 de 32 bits (AAPPLEBY, 2026). A escolha desta função não-criptográfica alinha-se com o estado da arte do setor de *Big Data*. Sendo o algoritmo de eleição de bibliotecas consagradas como a *libbloom*, o MurmurHash2 oferece uma velocidade de execução excelente e um efeito de avalanche (*avalanche effect*) adequado para a distribuição uniforme de chaves. A implementação tirará partido do parâmetro de "semente"(*seed*) da função: em vez de se programarem múltiplas funções *hash* distintas — o que introduziria complexidade desnecessária no código —, o sistema fará invocações iterativas ao mesmo algoritmo, alterando apenas a semente de inicialização com números primos, gerando assim as k posições independentes exigidas pela matemática do filtro. O uso exclusivo da versão de 32 bits evita o desperdício de ciclos de *clock* do CPU com blocos de 64 bits, uma vez que o espaço de endereçamento pós-módulo dos vetores planejados não excederá este limite.

Por fim, para assegurar a orquestração da experiência, a geração dos *datasets* e a análise estatística dos resultados, recorrer-se-á a *scripts* auxiliares desenvolvidos em Python 3. Esta linguagem, apoiada por bibliotecas de análise de dados (como *Pandas* e *NumPy*) e de visualização gráfica (como *Matplotlib*), será responsável por processar os registros de tempo (*logs*) e exportar os gráficos comparativos de desempenho, separando assim o ambiente crítico de execução em C do ambiente de análise de resultados.

4.3 ENGENHARIA DE DADOS: DATASETS E ESTRATÉGIA DE AMOSTRAGEM

A avaliação empírica de estruturas probabilísticas exige um ecossistema de dados que replique fielmente a heterogeneidade e a escala dos ambientes industriais de Big Data. Se os testes fossem realizados apenas com cadeias de caracteres (strings) simples e sequenciais, os resultados de dispersão das funções hash seriam artificialmente ideais, mascarando o comportamento real dos algoritmos e subestimando a taxa de colisões. Por conseguinte, esta pesquisa adotará uma estratégia combinada de engenharia de dados, utilizando conjuntos de dados (datasets) que mesclam chaves sintéticas geradas por algoritmos e dados estruturados baseados em formatos regulamentados do mundo real.

4.3.1 Natureza e Tipologia das Chaves de Teste

Para testar a resiliência e a capacidade de espalhamento (*hash*) uniforme da função MurmurHash2 sob diferentes níveis de entropia e comprimentos de entrada, o ambiente experimental será alimentado por três tipologias distintas de chaves:

1. Identificadores Únicos Universais (UUIDs): Consistem em strings de tamanho fixo (36 caracteres) com altíssima aleatoriedade nativa. A escolha de UUIDs visa simular chaves primárias de bancos de dados modernos NoSQL (como MongoDB ou Cassandra). Devido à sua alta entropia, este conjunto de dados servirá como o cenário ideal para observar o comportamento padrão dos filtros sob condições teóricas de dispersão quase perfeita.

2. Formatos Estruturados Numéricos (*CPFs e CNPJs*): Serão geradas massas de dados contendo sequências numéricas que simulam estritamente os padrões de Cadastro de Pessoas Físicas (11 dígitos) e Cadastro Nacional da Pessoa Jurídica (14 dígitos). Diferente dos UUIDs, estes formatos possuem baixa entropia e forte correlação estrutural (repetição de blocos iniciais dependendo da região geográfica de emissão). Este teste é crucial: ele forçará a função hash a provar o seu efeito de avalanche (avalanche effect), demonstrando se ela é capaz de separar e espalhar uniformemente chaves que são textualmente muito semelhantes, evitando o agrupamento (clustering) no vetor de bits.

3. Localizadores Uniformes de Recursos (URLs): Conjunto composto por endereços de sites da internet. As URLs apresentam tamanho altamente variável e partilham prefixos idênticos frequentes (como http://www, https://, .com, .br). Este cenário replica o uso clássico do Filtro de Bloom em sistemas de segurança de rede e proxies (como o Google Safe Browsing), testando a eficiência e o tempo de processamento dos filtros perante entradas de comprimentos heterogêneos.

4.3.2 Mecanismo de Partição e Protocolo Experimental

O aspecto mais crítico da engenharia de dados deste trabalho reside no protocolo de separação das amostras. Para que a validação estatística da Taxa de Falsos Positivos (FPR) seja fidedigna, é metodologicamente proibido testar o filtro consultando aleatoriamente qualquer dado disponível no mesmo dataset. O volume total de chaves geradas para a experiência será fragmentado por um script orquestrador em duas macro-partições mutuamente disjuntas:

- Dataset de Inserção ('Dins'): Esta partição conterá o volume fixado de chaves (N) que serão efetivamente processadas pelas funções *hash* e gravadas dentro de cada variante do Filtro de Bloom e na tabela de referência determinística durante a fase de construção (*build phase*).

- Dataset de Consulta ('Dcons'): Esta partição será utilizada exclusivamente na fase de verificação (*lookup phase*) e será rigorosamente subdividida em duas categorias controladas:

  - Subconjunto de Chaves Existentes ('Dexist'): Composto por uma amostragem extraída diretamente de dentro de 'Dins'. Ao consultar estas chaves, o sistema espera uma precisão de 100% em todos os filtros. Qualquer retorno negativo aqui acusará

um erro gravíssimo de engenharia (Falso Negativo), servindo para validar a integridade lógica do código escrito em C.

– Subconjunto de Chaves Inexistentes ('Dinexist'): Composto por chaves inéditas que pertencem ao mesmo domínio (ex: se o filtro foi populado com CPFs, este subconjunto conterá CPFs válidos, mas que nunca foram inseridos em 'Dins'). É a execução deste bloco que permitirá contabilizar os Falsos Positivos reais: cada vez que um filtro probabilístico responder "Sim"para uma chave pertencente a 'Dinexist', o sistema incrementará o contador de erros práticos, permitindo calcular a divergência exata entre a probabilidade matemática teórica e a realidade computacional das soluções desenvolvidas.

4.4 PARÂMETROS DE TESTE E MÉTRICAS DE AVALIAÇÃO

Para quantificar de forma objetiva o compromisso (trade-off) entre a precisão da indexação e o consumo de recursos computacionais, o ambiente de testes operará sob um conjunto rigoroso de parâmetros de controle e extrairá métricas de desempenho em tempo de execução. O objetivo é ultrapassar a análise puramente teórica e recolher evidências empíricas sobre o comportamento das estruturas sob stress de I/O (Input/Output).

4.4.1 Parâmetros de Configuração (Variáveis Independentes)

A instanciação de cada variante do Filtro de Bloom basear-se-á em parâmetros matemáticos fundamentais fornecidos antecipadamente ao orquestrador em C. Para assegurar a equidade na comparação, todos os modelos partilharão as mesmas condições iniciais:

- Capacidade Estimada (n): O volume total de elementos únicos pertencentes ao dataset de inserção ('Dins'). Os testes escalarão este valor progressivamente (por exemplo, de 1 milhão para 10 milhões de registros) para observar a degradação da performance.
- Probabilidade de Erro Alvo (p): A taxa teórica de falsos positivos máxima tolerada pelo sistema (como 0,01 para 1

A partir das variáveis n e p, o sistema calculará, em tempo de compilação ou inicialização, as variáveis dependentes da estrutura: o tamanho exato do vetor de memória (m, alocado em bits ou variáveis de 32 bits, dependendo do modelo) e o número ótimo de invocações independentes da função MurmurHash2 (k).

4.4.2 Métricas de Desempenho (Variáveis Dependentes)

Durante as fases ativas de construção e verificação, o script de monitorização fará a coleta e o registro logístico dos seguintes indicadores de desempenho, que servirão de base para

a geração dos resultados finais:

1. Consumo Efetivo de Memória (Pegada Espacial): Medido em Kilobytes (KB) e Megabytes (MB). Esta métrica confrontará a alocação de memória primária exigida pelos vetores/matrizes do Filtro de Bloom contra o peso estrutural do HashSet determinístico (a linha de base). Esta medição provará, de forma tangível, a viabilidade da estrutura probabilística para evitar o esgotamento de memória (gargalo de espaço) em ambientes de Big Data.

2. Taxa de Falsos Positivos Prática (FPR Prática): O indicador de exatidão mais crítico do estudo. Será calculada através da fração entre o número de erros reais registados (quando o filtro responde positivamente a uma chave do subconjunto de chaves inexistentes 'Dinexist') e o volume total de consultas desse subconjunto. A FPR prática será matematicamente cruzada com a probabilidade teórica (p) para validar a qualidade da dispersão do algoritmo MurmurHash2 e identificar a ocorrência de agrupamentos espúrios (*clustering*).

3. Latência de Inserção (*Build Time*): O tempo total de processamento no CPU, cronometrado rigorosamente em milissegundos (ms), necessário para o sistema processar os cálculos de espalhamento e gravar o vetor completo a partir do dataset de inserção. Esta métrica evidenciará o impacto computacional da variante r-Dimensional (rDBF), que propõe reduzir as chamadas de hash.

4. Latência de Verificação (*Lookup Time*): O tempo despendido para consultar e validar as chaves (tanto do 'Dexist' quanto do 'Dinexist'). Espera-se comprovar que, através da utilização rigorosa de operações bitwise na linguagem C, a arquitetura alcance uma complexidade temporal O(k) constante e de latência ultrabaixa, mitigando assim os tradicionais gargalos de processamento.

5. Taxa de Colisões Estruturais: Uma métrica secundária, focada nas variantes modificadas (como o modelo particionado e o híbrido), concebida para contabilizar o número de vezes que chaves distintas sobrescrevem exatamente as mesmas posições de memória, fornecendo dados analíticos para futuras otimizações de arquitetura de software.

5 CONSIDERAÇÕES FINAIS

Este capítulo encerra a primeira etapa desta pesquisa, correspondente ao Trabalho de Conclusão de Curso I (TCC I). Nele, apresenta-se uma síntese das atividades desenvolvidas, os resultados que se almeja alcançar na próxima etapa e, fundamentalmente, o plano de ação operacionalizado por meio de um cronograma de atividades projetado para a execução do Trabalho de Conclusão de Curso II (TCC II).

A jornada percorrida no TCC I concentrou-se na fundamentação teórica e na concepção arquitetural de uma solução baseada em Filtros de Bloom para a mitigação de custos computacionais em operações de entrada e saída (I/O) em bancos de dados. A pesquisa bibliográfica permitiu compreender as limitações das varreduras lineares exaustivas e o comportamento probabilístico de estruturas de dados de *hash*, culminando na escolha das variantes do Filtro de Bloom como objetos empíricos de estudo. O projeto metodológico, detalhado no Capítulo 4, estabeleceu o rigor e as métricas de avaliação — como a Taxa de Falsos Positivos (FPR) e as latências de inserção e verificação — necessários para validar a eficiência da proposta.

Portanto, os objetivos propostos para esta fase inicial foram satisfeitos. O entendimento matemático das falhas latentes da literatura — como a mecânica real do *overflow* em filtros de contagem — e o mapeamento dos riscos do modelo híbrido proposto pavimentam o caminho para uma transição segura rumo à fase de implementação, experimentação e análise estatística.

5.1 RESULTADOS ESPERADOS

Com a consolidação da proposta metodológica, a execução prática deste projeto (a ser realizada na segunda etapa do Trabalho de Conclusão de Curso) visa validar empiricamente a fundamentação teórica construída. Espera-se que a implementação da arquitetura em linguagem C demonstre:

- Uma redução substancial no tempo de acesso a dados inexistentes, validando a eficácia do Filtro de Bloom como uma camada de triagem de alta velocidade (*gatekeeper*) anterior às consultas em disco ou bancos de dados relacionais;
- A confirmação da complexidade temporal $O(k)$ constante para a validação de chaves, mesmo sob cenários de estresse de dados em larga escala;
- A convergência entre a Taxa de Falsos Positivos (FPR) prática e a teórica, atestando a robustez da distribuição uniforme fornecida pela função de espalhamento *MurmurHash2*;

- O mapeamento preciso dos limites de viabilidade do Modelo Probabilístico Híbrido, identificando se a sua união de características resulta em ganho real de desempenho ou em degradação por sobreposição de *trade-offs*.

5.2 CRONOGRAMA DE ATIVIDADES PARA O TCC II

Para que os objetivos propostos sejam alcançados de forma sistemática, as atividades da próxima etapa (TCC II) foram estruturadas em um cronograma de execução. O planejamento engloba desde a codificação inicial até a defesa pública do trabalho, conforme detalhado na Tabela 2.

Tabela 2 – Cronograma de atividades previstas para o TCC II.

Atividade / Mês	**Ago**	**Set**	**Out**	**Nov**	**Dez**
1. Implementação dos algoritmos base (Filtro Padrão e de Contagem)	X				
2. Implementação das variantes (Particionado e R-Dimensional)	X	X			
3. Implementação do Modelo Híbrido e Concorrência (*pthreads*)		X	X		
4. Integração da Linha de Base (`uthash`) e *Datasets*		X	X		
5. Execução dos testes de *benchmark* e medição de latências			X	X	
6. Análise dos resultados estatísticos (FPR, colisões e CPU)				X	
7. Escrita dos capítulos de Resultados, Discussões e Conclusão				X	X
8. Revisão final do documento, formatação ABNT e Defesa Pública					X

Fonte: O autor (2026).

REFERÊNCIAS

AAPPLEBY. **smhasher**. 2026. Disponível em: <https://github.com/aappleby/smhasher>. Acesso em: 15 jun. 2026.

ABDENNEBI, A.; KAYA, K. A bloom filter survey: Variants for different domain applications. **ArXiv**, abs/2106.12189, 2021. Disponível em: <https://arxiv.org/abs/2106.12189>.

BLOOM, B. H. Space/time trade-offs in hash coding with allowable errors. **Communications of the ACM**, v. 13, n. 7, p. 422–426, 1970.

BRODER, A.; MITZENMACHER, M. Network applications of bloom filters: A survey. **Internet Mathematics**, v. 1, n. 4, p. 485–509, 2004.

GOOGLE. **Sobre os filtros do Bloom**. 2026. Disponível em: <https://docs.cloud.google.com/memorystore/docs/valkey/about-bloom-filters?hl=pt-br>. Acesso em: 20 jun. 2026.

JÜRGENS, M.; LENZ, H.-J. Tree based indexes versus bitmap indexes: A performance study. In: **Proceedings of the International Workshop on Design and Management of Data Warehouses (DMDW'99)**. Heidelberg, Germany: CEUR-WS, 1999. Disponível em: <http://sunsite.informatik.rwth-aachen.de/Publications/CEUR-WS/Vol-19/>. Acesso em: 29 jun. 2026.

JVIRKKI. **libbloom**. 2022. Disponível em: <https://github.com/jvirkki/libbloom>. Acesso em: 15 jun. 2026.

MEDINA, J. M.; BARRANCO, C. D.; PONS, O. Indexing techniques to improve the performance of necessity-based fuzzy queries using classical indexing of rdbms. **Fuzzy Sets and Systems**, v. 351, p. 90–107, 2018.

NAYAK, S.; PATGIRI, R.; BORAH, A. A review on impact of bloom filter on named data networking: The future internet architecture. **ArXiv**, abs/2005.06965, 2020. Disponível em: <https://arxiv.org/abs/2005.06965>.

OOSTERHUIS, H.; CULPEPPER, J. S.; RIJKE, M. de. The potential of learned index structures for index compression. In: **Proceedings of the 23rd Australasian Document Computing Symposium**. New York, NY, USA: Association for Computing Machinery, 2018.

PATGIRI, R. robustbf: A high accuracy and memory efficient 2d bloom filter. **ArXiv**, https://arxiv.org/abs/2106.04365, 2021.

PATGIRI, R.; NAYAK, S.; BORGOHAIN, S. K. Hunting the pertinency of bloom filter in computer networking and beyond: A survey. **Journal of Computer Networks and Communications**, v. 2019, n. 1, p. 2712417, 2019.

PATGIRI, R.; NAYAK, S.; MUPPALANENI, N. B. **Bloom Filter: A Data Structure for Computer Networking, Big Data, Cloud Computing, Internet of Things, Bioinformatics, and Beyond**. London: Academic Press, 2023.

REDIS. **Bloom filter**. 2022. Disponível em: <https://redis.io/docs/latest/develop/data-types/probabilistic/bloom-filter/>. Acesso em: 22 jun. 2026.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020. ISBN 9788595157330.

TARKOMA, S.; ROTHENBERG, C. E.; LAGERSPETZ, E. Theory and practice of bloom filters for distributed systems. **IEEE Communications Surveys & Tutorials**, v. 14, n. 1, p. 131–155, 2012.

TROYDHANSON. **uthash**. 2026. Disponível em: <https://github.com/troydhanson/uthash>. Acesso em: 16 jun. 2026.

ZOBEL, J.; MOFFAT, A.; RAMAMOHANARAO, K. Inverted files versus signature files for text indexing. **ACM Transactions on Database Systems**, v. 23, n. 4, p. 453–490, 1998.

APÊNDICE A – DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

O autor utilizou as ferramentas Consensus[1] e SciSpace[2] na busca de artigos e livros que fundamentaram a fundamentação teórica, abrangendo estudos sobre estruturas probabilísticas de dados (*Bloom Filters*), funções de *hash* e otimização de consultas em Sistemas Gerenciadores de Bancos de Dados (SGBDs). Após o uso das ferramentas, o autor revisou e editou o conteúdo em conformidade com o método científico e assume total responsabilidade pelo conteúdo da publicação.

Na construção do referencial teórico, a partir da análise dos artigos e livros selecionados, o autor empregou a ferramenta NotebookLM[3], que facilitou a localização de informações relevantes para a redação. Após o uso da ferramenta, o autor revisou e editou o conteúdo em conformidade com o método científico e assume total responsabilidade pelo conteúdo da publicação.

Na correção e na revisão do texto escrito, o autor recorreu à ferramenta Gemini Pro[4], da Google, com vistas ao aprimoramento da clareza, da coesão e da correção gramatical. Após o uso da ferramenta, o autor revisou e editou o conteúdo em conformidade com o método científico e assume total responsabilidade pelo conteúdo da publicação.

No auxílio à construção de tabelas, figuras e quadros em LaTeX, o autor utilizou a ferramenta OpenCode Big Pickle. Após o uso da ferramenta, o autor revisou e editou o conteúdo em conformidade com o método científico e assume total responsabilidade pelo conteúdo da publicação.

---

1 <https://consensus.app/>
2 <https://scispace.com/>
3 <https://notebooklm.google.com/>
4 <https://gemini.google.com/>